GitOps Lab — Layer by Layer Guide

This document explains how the lab is built, one layer at a time, from the laptop all the way down to the running services.

---

1. Overview

The lab provisions and configures a 3-VM environment entirely through code:

- **OpenTofu** creates the VMs (Infrastructure as Code)
- **Ansible** installs and configures software on those VMs (Configuration Management)
- Nothing is clicked by hand in the Proxmox UI — every VM, every package, every config file is created by a tool reading a file

This pattern — describing infrastructure declaratively and applying it with automation — is what "GitOps" refers to here: the Git repository is the single source of truth for what the infrastructure should look like.

---

2. Layer 1 — Host machine (Windows + VMware)

A Windows laptop runs **VMware Workstation**, which hosts a single VM running **Proxmox VE**. This is called *nested virtualization* — a hypervisor running inside another hypervisor's VM.

VMware provides a NAT network for everything inside it:

Setting	Value
Network	192.168.72.0/24
Gateway	192.168.72.2

Every VM Proxmox creates lives inside this network range.

Why nested virtualization: it lets the entire lab run inside a single VM on a laptop, with no dedicated server hardware required. The tradeoff is reduced performance and an extra layer of complexity.

---

3. Layer 2 — Hypervisor (Proxmox VE)

Proxmox is the hypervisor that actually runs the lab's VMs. Key concepts used in this lab:

- **Storage**: `local-lvm`, an LVM-thin pool. Thin provisioning means disks report a size larger than what's physically allocated upfront — space is only consumed as data is actually written.
- **Template**: a Ubuntu 24.04 cloud image was converted into a Proxmox template (VM ID 9000). Templates can be cloned quickly instead of installing an OS from scratch each time.
- **Cloud-init**: handles first-boot configuration of cloned VMs — setting hostname, network, SSH keys, and user accounts without manual setup.

---

4. Layer 3 — Provisioning (OpenTofu)

OpenTofu is an open-source fork of Terraform. It reads `.tf` files describing the desired infrastructure state and creates/updates/destroys resources to match.

What it provisions

```
locals {
  vms = {
    "vm-web-01" = { vm_id = 101, cores = 1, memory = 1024, ip = "192.168.72.11/24" }
    "vm-db-01"  = { vm_id = 102, cores = 2, memory = 2048, ip = "192.168.72.12/24" }
    "vm-mon-01" = { vm_id = 103, cores = 2, memory = 2048, ip = "192.168.72.13/24" }
  }
}
```

Each VM is cloned from the template, given a static IP via cloud-init, and configured with an SSH key for passwordless access:

```
initialization {
  datastore_id = "local-lvm"
  user_account {
    username = "ubuntu"
    keys     = [trimspace(file("~/.ssh/id_ed25519.pub"))]
  }
  ip_config {
    ipv4 {
      address = each.value.ip
      gateway = var.vm_network_gateway
    }
  }
}
```

Key lesson: declarative vs imperative

Instead of manually clicking "create VM" three times in the Proxmox UI, this `locals` block is the single source of truth. Running `tofu apply` again after changing a value (e.g. bumping `vm-db-01`'s RAM) will only change what's different — it won't recreate VMs that already match the desired state.

Commands

```
cd terraform
tofu init      # downloads the Proxmox provider plugin
tofu plan      # shows what would change, without applying
tofu apply     # creates/updates the VMs
```

---

5. Layer 4 — Configuration management (Ansible)

Once the VMs exist, Ansible takes over to install and configure software on them.

Why Ansible runs from inside a VM, not the laptop

Ansible's core architecture relies on Unix-style SSH and shell execution — it does not support Windows as a control node. Attempting to run it on Windows fails with `OSError: [WinError 1] Incorrect function`.

Solution: `vm-web-01` was designated as the Ansible control node. The SSH private key was copied there, and Ansible runs from inside that VM, connecting out to all three VMs (including itself) over SSH.

Inventory

`inventory/hosts.yml` groups the VMs by role so playbooks can target specific groups:

```
all:
  vars:
    ansible_user: ubuntu
    ansible_ssh_private_key_file: ~/.ssh/id_ed25519
```

```
  children:
    webservers:
      hosts:
        vm-web-01:
          ansible_host: 192.168.72.11
    databases:
      hosts:
        vm-db-01:
          ansible_host: 192.168.72.12
    monitoring:
      hosts:
        vm-mon-01:
          ansible_host: 192.168.72.13
```

Roles

Ansible organizes configuration into reusable **roles**, each with its own `tasks/`, `handlers/`, and `templates/` directories.

Role	Applies to	Installs
`common`	all VMs	apt updates, UFW firewall, timezone, deploy user
`webserver`	vm-web-01	Nginx, custom server block via Jinja2 template
`database`	vm-db-01	PostgreSQL
`monitoring`	vm-mon-01	Prometheus, Grafana

Handlers are tasks that only run when triggered by a `notify` — used here so Nginx only reloads when its config actually changes, not on every playbook run.

Commands

```
cd ansible
ansible all -m ping             # verify connectivity to all hosts
ansible-playbook site.yml       # apply all roles
```

---

6. Layer 5 — The services

Service	Runs on	Port	Purpose
Nginx	vm-web-01	80, 443	Web server, serves a sample page and `/health` endpoint
PostgreSQL	vm-db-01	5432	Relational database
Prometheus	vm-mon-01	9090	Metrics collection
Grafana	vm-mon-01	3000	Metrics visualization (default login: admin/admin)

Each VM also runs **UFW** (Uncomplicated Firewall), configured by the `common` role to allow only SSH plus whatever ports that VM's role needs.

---

7. Repository structure

```
gitops-lab/
├── terraform/
```

```
│   ├── main.tf
│   ├── providers.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── .gitignore          # excludes tfvars, tfstate, .terraform/
└── ansible/
    ├── ansible.cfg
    ├── site.yml
    ├── inventory/
    │   └── hosts.yml
    └── roles/
        ├── common/
        │   └── tasks/main.yml
        ├── webserver/
        │   ├── tasks/main.yml
        │   ├── handlers/main.yml
        │   └── templates/nginx.conf.j2
        ├── database/
        │   ├── tasks/main.yml
        │   └── handlers/main.yml
        └── monitoring/
            ├── tasks/main.yml
            └── templates/prometheus.yml.j2
```